

Apache Iceberg Data

模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：iceberg-data (org.apache.iceberg.data)

版本：Apache Iceberg 1.9.2 (tag: apache-iceberg-1.9.2)

生成工具：AI Architecture Analyzer

生成日期：2026年03月24日

第1章 模块概述

1.1 模块定位

Apache Iceberg Data 模块 (iceberg-data) 是 Iceberg 表格式项目中的通用数据读写层，位于 Iceberg Core 之上、计算引擎集成层 (Spark、Flink 等) 之下。该模块的核心使命是提供一套与计算引擎无关的、基于通用 Record 类型的数据读取和写入能力，同时完整支持 Iceberg V2 规范中的行级删除 (Row-Level Deletes) 语义。

iceberg-data 模块包含 14 个主源码文件，全部位于 org.apache.iceberg.data 包下，结构扁平、职责清晰。它为上层引擎集成层提供了可复用的抽象基类和工厂模式框架，同时也可以独立使用（如在单元测试或命令行工具中直接读写 Iceberg 表）。

1.2 核心能力

- 通用数据读取 — 提供面向用户的 IcebergGenerics API，支持 Builder 模式配置表扫描参数
- 行级删除处理 — 完整支持 Equality Deletes、Position Deletes 和 Deletion Vectors (DV)
- 多格式写入 — 通过抽象工厂模式支持 Avro、Parquet、ORC 三种文件格式的数据和删除文件写入
- 格式注册机制 — 基于 FormatModelRegistry 的插件化格式读写器注册
- 表迁移工具 — 支持从 Hive/Hadoop 表迁移到 Iceberg 表
- 类型转换 — Java 时间类型与 Iceberg 内部类型之间的透明转换

1.3 源码文件清单

文件名	行数	类型	职责描述
IcebergGenerics.java	104	入口 API	面向用户的通用读取 API，Builder 模式
DeleteLoader.java	47	接口	删除文件加载 API 契约
BaseDeleteLoader.java	286	实现类	DeleteLoader 默认实现，支持 EQ/POS 删除和 DV
DeleteFilter.java	331	抽象类	行级删除过滤核心逻辑（位置删除+等值删除）
GenericDeleteFilter.java	53	实现类	DeleteFilter 的 Record 类型具体实现
BaseFileWriterFactory.java	376	抽象类	（已废弃）传统多格式文件写入器工厂基类
RegistryBasedFileWriterFactory.java	183	抽象类	基于注册表的新一代文件写入器工厂基类
GenericFileWriterFactory.java	400	实现类	基于注册表的通用 Record 写入器工厂 + Builder
GenericAppenderFactory.java	351	实现类	（已废弃）通用 FileAppender 工厂

文件名	行数	类型	职责描述
GenericFormatModels.java	69	工具类	Avro/Parquet/ORC 格式读写器注册
GenericReader.java	124	包内类	通用数据读取器，处理文件扫描任务
TableScanIterable.java	51	包内类	表扫描结果可迭代包装器
InternalRecordWrapper.java	115	适配器	Record 到 StructLike 的类型转换适配器
TableMigrationUtil.java	272	工具类	Hive/Hadoop 到 Iceberg 表迁移工具

第2章 核心架构设计

2.1 整体分层架构

iceberg-data 模块采用经典的三层架构设计：API 层提供面向用户的入口接口，核心层包含删除处理和数据读取的主要业务逻辑，基础设施层通过格式注册表与底层文件 I/O 交互。

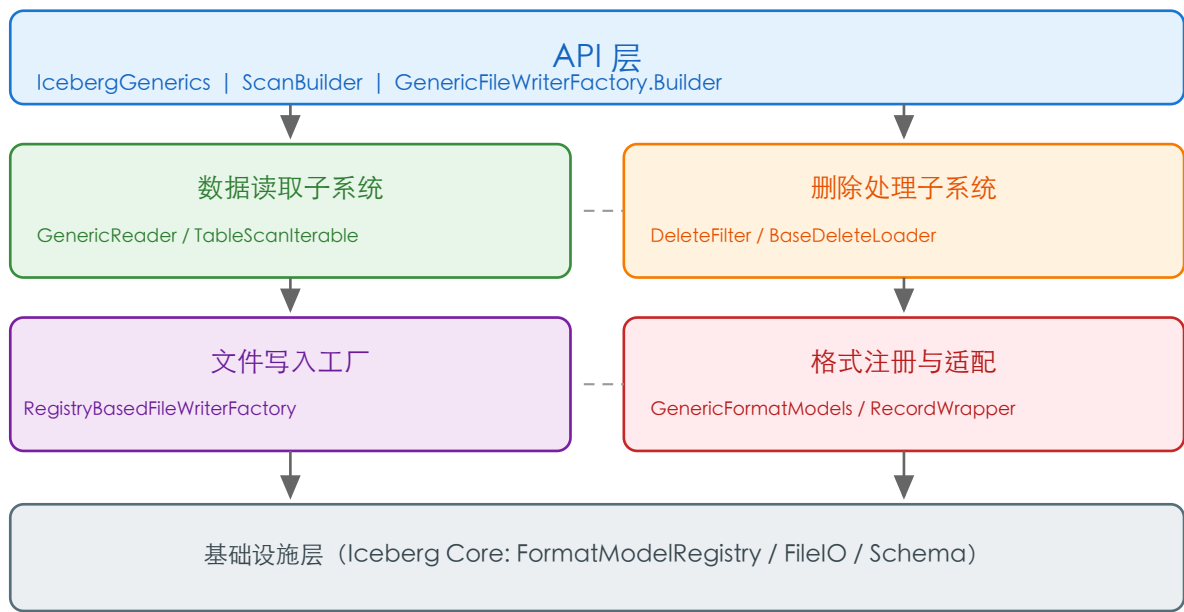


图2-1 iceberg-data 模块整体分层架构图

2.2 类继承关系图

模块内的类关系可以归纳为四条继承链和一个组合关系网：

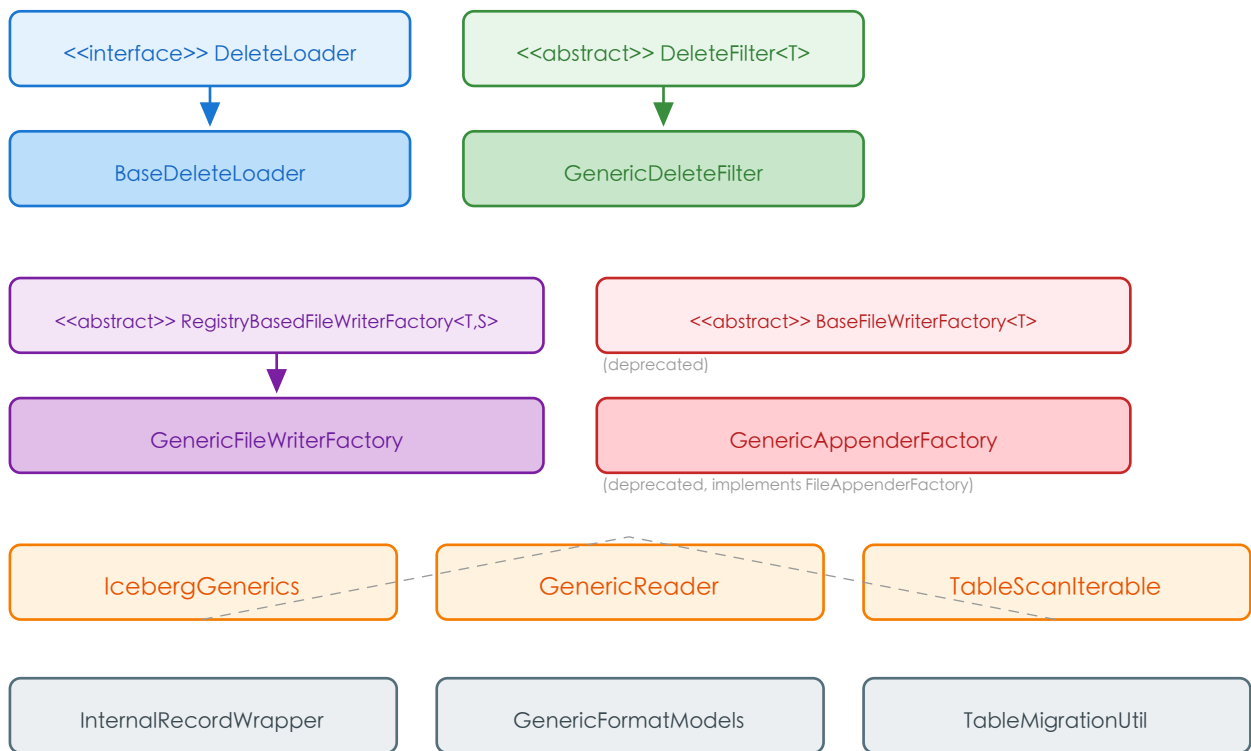


图2-2 iceberg-data 模块类继承与关系图

2.3 外部依赖关系

iceberg-data 模块依赖 Iceberg Core 提供的以下关键接口和类：

外部接口/类	所在模块	用途
FileWriterFactory	iceberg-core (io)	文件写入器工厂接口，定义 newDataWriter / newEqualityDeleteWriter / newPositionDeleteWriter
FileAppenderFactory	iceberg-core (io)	（已废弃）文件追加器工厂接口
FormatModelRegistry	iceberg-core (formats)	全局格式模型注册表，管理 Avro/Parquet/ORC 读写器
CloseableIterable	iceberg-core (io)	可关闭的迭代器接口，管理资源生命周期
StructLike	iceberg-core	Iceberg 通用结构化数据接口
Record	iceberg-core (data)	通用记录类型，实现 StructLike
PositionDeleteIndex	iceberg-core (deletes)	位置删除索引，支持 Roaring Bitmap
StructLikeSet	iceberg-core (util)	基于 StructLike 的集合，用于等值删除匹配
Table / TableScan	iceberg-core	表抽象和扫描规划接口

第3章 核心类设计模式详解

3.1 IcebergGenerics — Builder 模式

IcebergGenerics 是 iceberg-data 模块面向用户的主入口类，采用经典的 Builder 模式设计。私有构造函数防止直接实例化，通过静态方法 `read(Table)` 返回内部类 `ScanBuilder`，支持链式调用配置过滤条件、列投影、快照选择等扫描参数，最终通过 `build()` 方法生成 `CloseableIterable<Record>`。

设计意图：将复杂的表扫描配置过程与扫描执行分离，用户无需了解 `GenericReader` 和 `TableScanIterable` 的实现细节。

关键方法链：

```
IcebergGenerics.read(table) // ■■■■■■
.where(expression) // ■■■■■■
.select("col1", "col2") // ■■■■■■
.useSnapshot(snapshotId) // ■■■■■■
.caseInsensitive() // ■■■■■■
.reuseContainers() // ■■■■■■■■■■
.build() // ■■ TableScanIterable
```

3.2 RegistryBasedFileWriterFactory — 抽象工厂模式

RegistryBasedFileWriterFactory 实现了 抽象工厂模式，它是 `FileWriterFactory<T>` 接口的抽象实现，通过 `FormatModelRegistry` 注册表动态获取不同文件格式（Avro、Parquet、ORC）的写入器构建器。子类 `GenericFileWriterFactory` 将泛型 `T` 绑定为 `Record` 类型。

设计意图：将写入器的创建过程与具体文件格式解耦。新增文件格式只需在 `FormatModelRegistry` 中注册，无需修改工厂代码。同时通过 Builder 内部类提供流式配置 API。

产品族：

- `DataWriter<T>` — 数据文件写入器
- `EqualityDeleteWriter<T>` — 等值删除文件写入器
- `PositionDeleteWriter<T>` — 位置删除文件写入器

3.3 DeleteFilter — 模板方法模式

`DeleteFilter<T>` 是一个 模板方法模式 的经典实现。它在抽象类中定义了行级删除过滤的完整算法骨架（`filter` → `applyPosDeletes` → `applyEqDeletes`），同时将类型转换等步骤延迟到子类 `GenericDeleteFilter` 实现：

模板方法（定义在基类中）：

- `filter(records)` — 算法骨架：先应用位置删除，再应用等值删除

- `applyPosDeletes(records)` — 通过 `PositionDeleteIndex` 过滤被删除位置的行
- `applyEqDeletes(records)` — 通过 `StructLikeSet` 匹配等值删除条件

抽象方法（子类实现）：

- `asStructLike(record)` — 将 `Record` 转换为 `StructLike` 以便比较
- `getInputFile(location)` — 加载删除文件的 `InputFile`

3.4 InternalRecordWrapper — 适配器模式

`InternalRecordWrapper` 是一个 适配器（Adapter）模式 的实现，它将 `Record` 对象适配为 `StructLike` 接口，同时完成 Java 时间类型到 Iceberg 内部类型的自动转换。

转换规则：

Java 类型	Iceberg 内部类型	转换方法
<code>LocalDate</code>	<code>int</code> (天数)	<code>DateTimeUtil.daysFromDate()</code>
<code>LocalTime</code>	<code>long</code> (微秒)	<code>DateTimeUtil.microsFromTime()</code>
<code>LocalDateTime</code>	<code>long</code> (微秒)	<code>DateTimeUtil.microsFromTimestamp()</code>
<code>OffsetDateTime</code>	<code>long</code> (微秒)	<code>DateTimeUtil.microsFromTimestamptz()</code>
<code>byte[]</code>	<code>ByteBuffer</code>	<code>ByteBuffer.wrap()</code>
<code>StructLike</code> (嵌套)	<code>StructLike</code>	递归包装

3.5 GenericFormatModels — 策略模式 + 注册表模式

`GenericFormatModels` 结合 策略模式 和 注册表模式，将三种文件格式（Avro、Parquet、ORC）的读写策略注册到全局 `FormatModelRegistry`。每种格式都是一个独立的策略实现，通过 `Lambda` 表达式提供 `Writer` 和 `Reader` 的创建逻辑。

注册项：

- `AvroFormatModel` — `DataWriter::create` / `PlannedDataReader::create`
- `ParquetFormatModel` — `GenericParquetWriter::create` / `GenericParquetReaders::buildReader`
- `ORCFormatModel` — `GenericOrcWriter::buildWriter` / `GenericOrcReader::buildReader`
- 各格式还注册了对应的 `PositionDelete` 特化模型

第4章 核心流程图

4.1 通用数据读取流程

用户通过 `IcebergGenerics.read(table)` 入口发起表数据读取，经历扫描规划、任务拆分、文件打开、删除过滤、残余过滤等步骤，最终返回干净的 `Record` 迭代器。

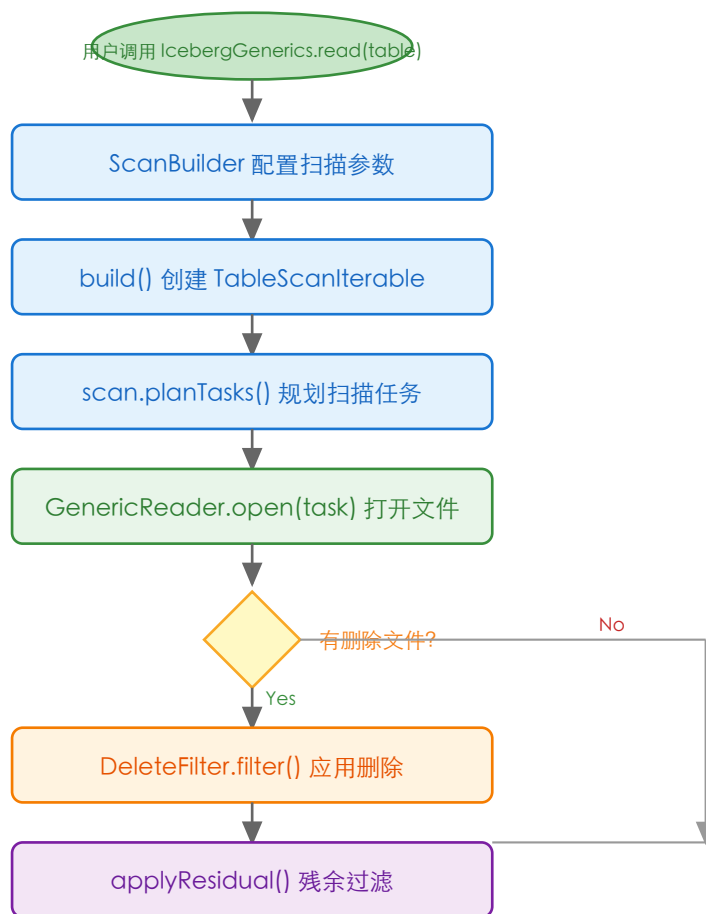


图4-1 通用数据读取流程图

4.2 行级删除处理流程

DeleteFilter.filter()

方法协调位置删除和等值删除两种机制，采用管道模式依次过滤。BaseDeleteLoader 负责从文件系统加载删除数据到内存数据结构。

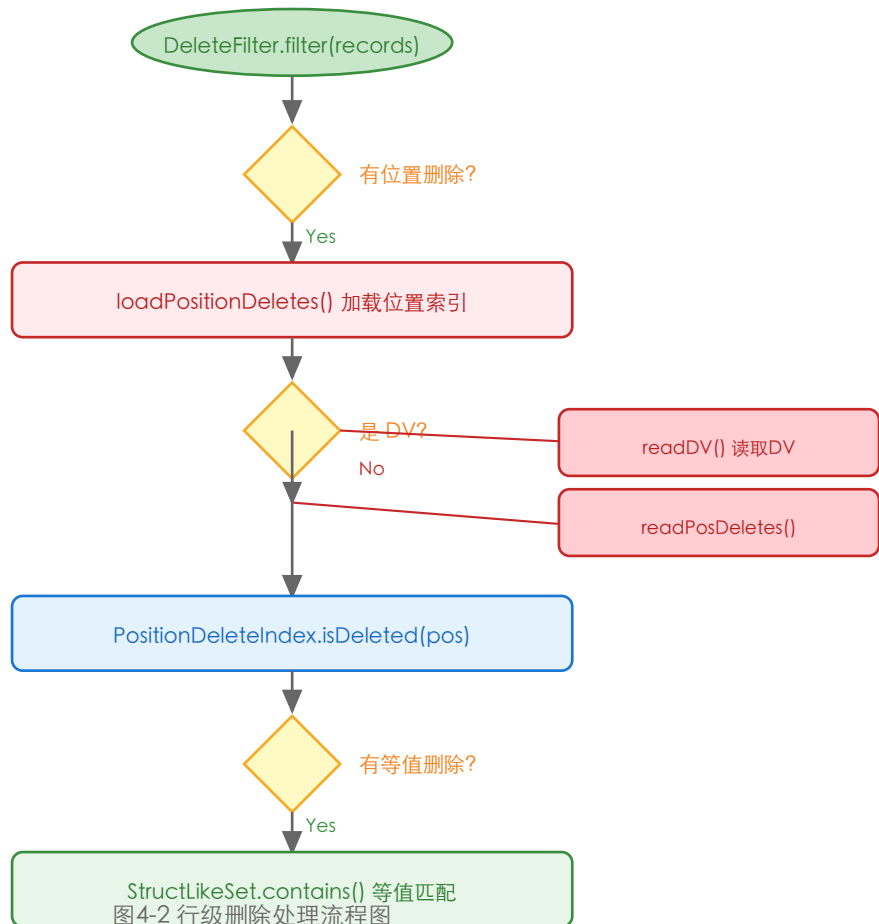


图4-2 行级删除处理流程图

4.3 文件写入器创建流程

GenericFileWriterFactory 通过 Builder 模式配置参数，调用 RegistryBasedFileWriterFactory 基类方法，最终由 FormatModelRegistry 根据文件格式动态创建具体的写入器。

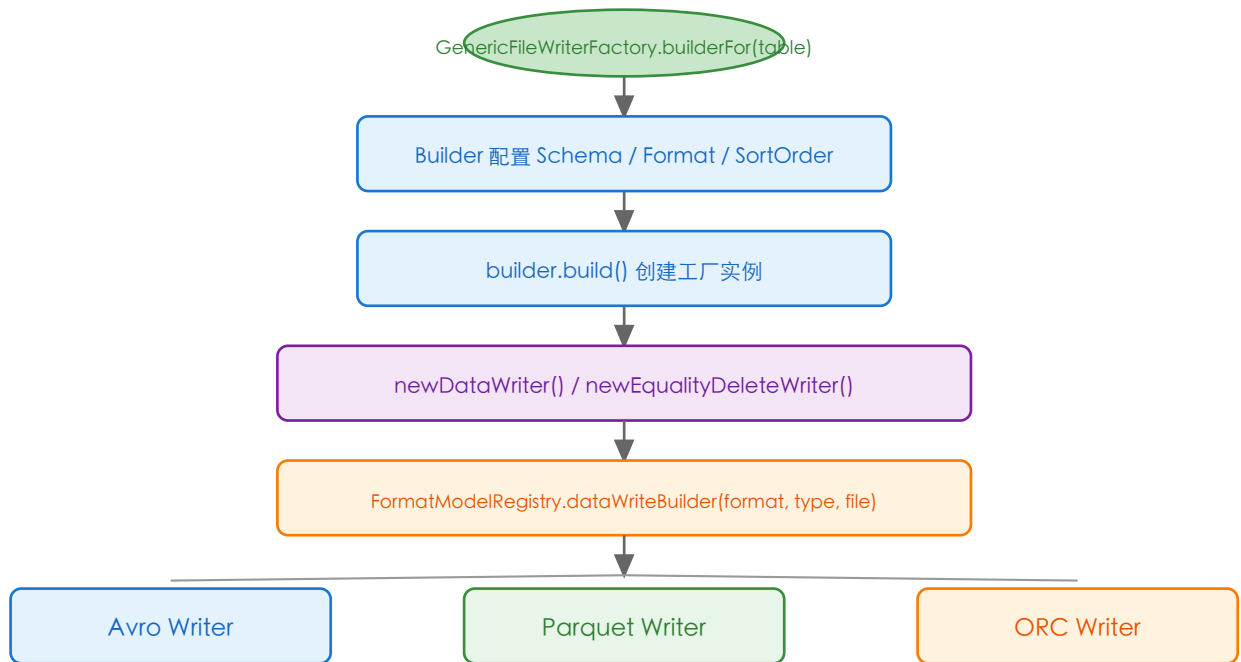


图4-3 文件写入器创建流程图

第5章 核心时序图

5.1 通用数据读取时序图

展示从用户调用 `IcebergGenerics.read()` 到获取第一条 `Record` 的完整调用链，涉及 5 个参与者。

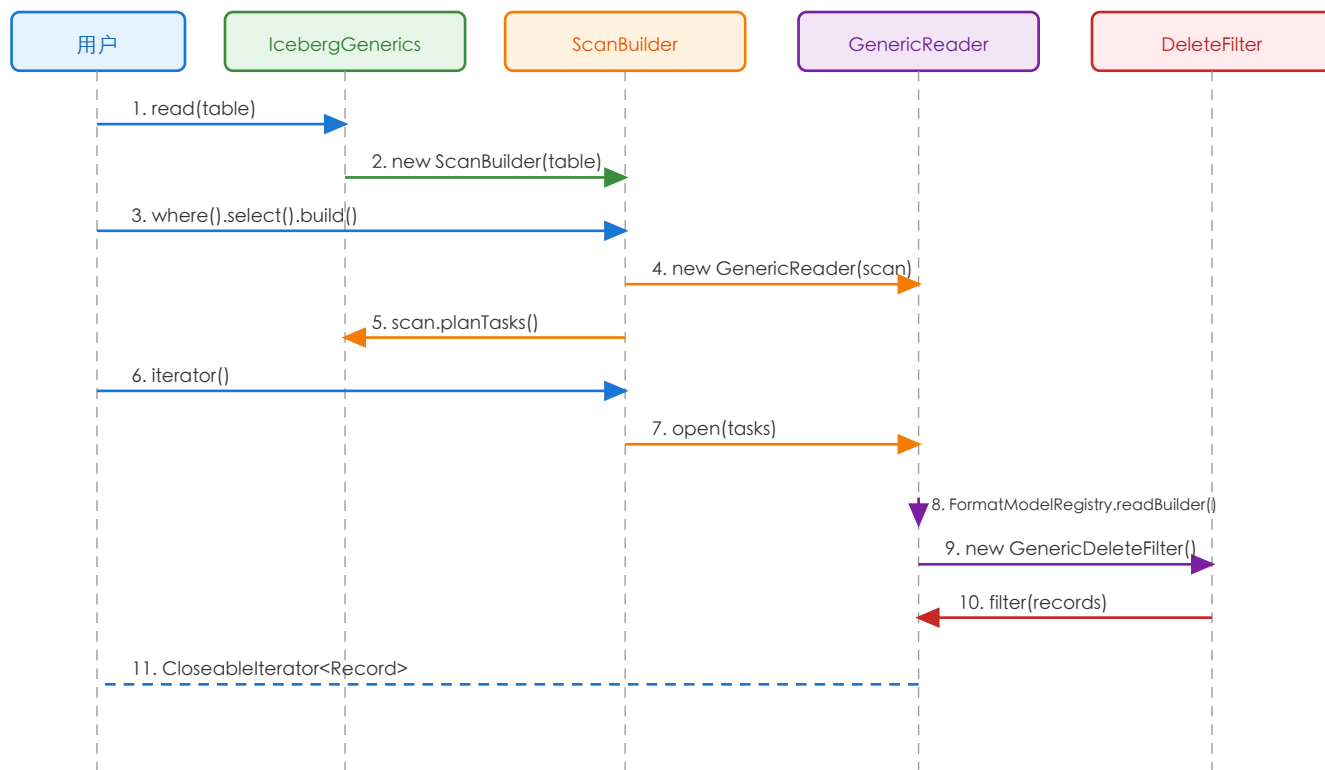


图5-1 通用数据读取时序图

5.2 删除文件加载时序图

展示 DeleteFilter 如何通过 BaseDeleteLoader 加载位置删除和等值删除数据。

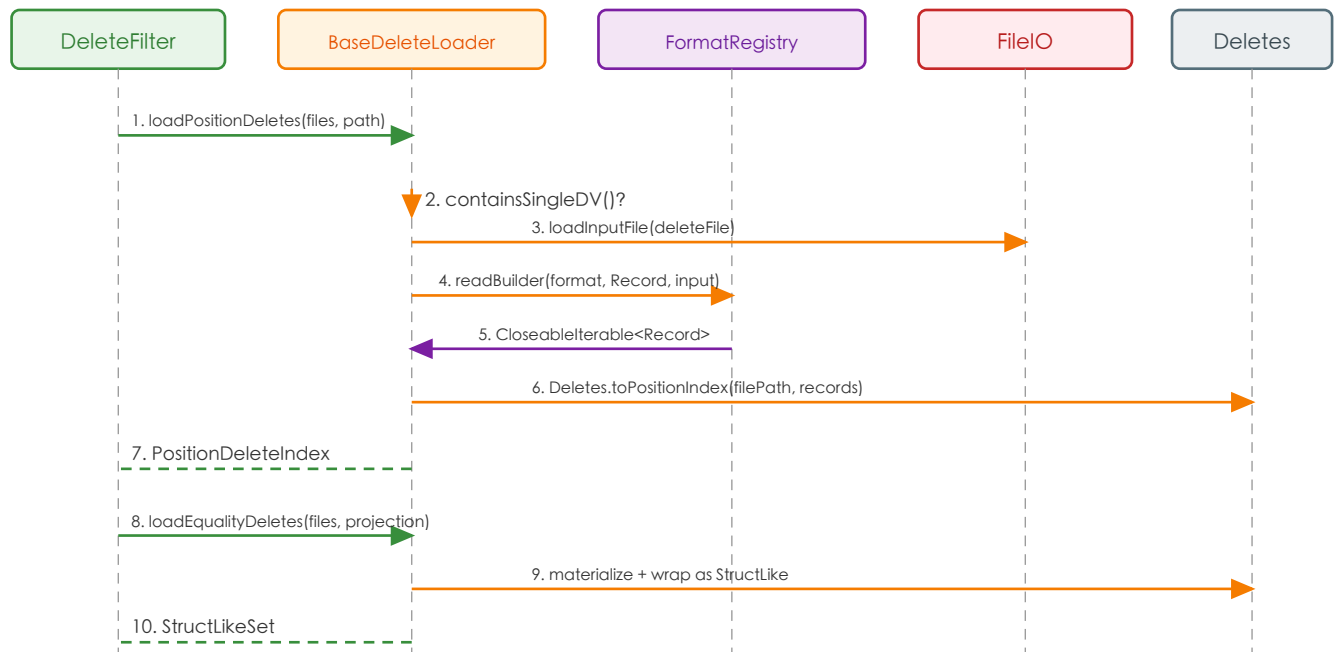


图5-2 删除文件加载时序图

第6章 关键场景调用链

6.1 场景一：表全量扫描读取

触发条件：用户通过 `IcebergGenerics.read(table).build()` 发起全表扫描

步骤	类.方法	说明
1	<code>IcebergGenerics.read(table)</code>	静态工厂方法，创建 <code>ScanBuilder</code>
2	<code>ScanBuilder.build()</code>	创建 <code>TableScanIterable</code> ，内部启动后台规划
3	<code>TableScanIterable.iterator()</code>	触发数据读取，调用 <code>GenericReader.open(tasks)</code>
4	<code>GenericReader.open(CombinedScanTask)</code>	展开为 <code>FileScanTask</code> 列表并逐个处理
5	<code>GenericReader.open(FileScanTask)</code>	创建 <code>GenericDeleteFilter</code> ，获取需要读取的 Schema
6	<code>GenericReader.openFile(task, schema)</code>	通过 <code>FormatModelRegistry.readBuilder</code> 打开文件
7	<code>DeleteFilter.filter(records)</code>	应用位置删除和等值删除过滤
8	<code>GenericReader.applyResidual()</code>	应用残余过滤表达式

6.2 场景二：位置删除文件加载（DV 模式）

触发条件：读取的数据文件关联了 Deletion Vector（DV）类型的删除文件

步骤	类.方法	说明
1	<code>DeleteFilter.deletedRowPositions()</code>	检查是否有位置删除文件
2	<code>BaseDeleteLoader.loadPositionDeletes(files, path)</code>	入口方法，判断是 DV 还是普通位置删除
3	<code>ContentFileUtil.containsSingleDV(files)</code>	检查是否为单个 DV 文件
4	<code>BaseDeleteLoader.validateDV(dv, filePath)</code>	校验 DV 的 offset/length/引用路径
5	<code>BaseDeleteLoader.readDV(dv)</code>	通过 <code>IOUtil.readFully</code> 读取 DV 字节数据
6	<code>PositionDeleteIndex.deserialize(bytes, dv)</code>	反序列化为 Roaring Bitmap 索引

6.3 场景三：等值删除文件加载与匹配

触发条件：读取的数据文件关联了 Equality Delete 类型的删除文件

步骤	类.方法	说明
1	DeleteFilter.applyEqDeletes()	按 equalityFieldIds 分组删除文件
2	Multimaps.newMultimap()	将相同字段集合的删除文件归组
3	BaseDeleteLoader.loadEqualityDeletes(files, schema)	加载所有等值删除记录
4	BaseDeleteLoader.openDeletes(file, projection)	通过 FormatModelRegistry 读取删除文件
5	InternalRecordWrapper.copyFor(record)	将 Record 包装为 StructLike
6	ImmutableList.copyOf()	物化到内存（便于缓存）
7	StructLikeSet.contains()	对每条数据行进行等值匹配

6.4 场景四：通用写入器创建

触发条件：引擎集成层需要创建数据文件写入器

步骤	类.方法	说明
1	GenericFileWriterFactory.builderFor(table)	创建 Builder 实例，从表属性推导默认格式
2	Builder.dataSchema(schema).build()	校验参数完整性，创建工厂实例
3	factory.newDataWriter(file, spec, partition)	调用基类 RegistryBasedFileWriterFactory
4	FormatModelRegistry.dataWriteBuilder(format, type, file)	从注册表获取格式特定的写入构建器
5	FileWriterBuilder.schema().metricsConfig().build()	链式配置后构建最终的 DataWriter

6.5 场景五：Hive 表迁移到 Iceberg

触发条件：用户调用 TableMigrationUtil 将已有 Hive 分区迁移到 Iceberg 表

步骤	类.方法	说明
1	TableMigrationUtil.listPartition(partition, uri, format, ...)	入口方法，列出分区中的文件
2	FileSystem.listStatus(partitionDir, HIDDEN_PATH_FILTER)	通过 Hadoop API 列出文件，过滤隐藏文件
3	Tasks.range(fileStatus.size()).executeWith(service)	并行处理文件列表
4	getParquetMetrics() / getOrcMetrics() / getAvroMetrics()	根据格式读取文件 footer 获取 Metrics

步骤	类.方法	说明
5	DataFiles.builder(spec).withMetrics(metrics).build()	构建 Iceberg DataFile 元数据

第7章 配置与扩展

7.1 关键配置属性

配置项	默认值	影响范围	说明
write.format.default	parquet	GenericFileWriterFactory	数据文件默认写入格式
write.delete.format.default	继承 write.format.default	GenericFileWriterFactory	删除文件默认写入格式
write.metadata.metrics.*	由 MetricsConfig 管理	所有写入器工厂	Metrics 收集配置

7.2 扩展点

iceberg-data 模块提供了多个扩展点供引擎集成层使用：

- DeleteFilter<T> — 抽象类，引擎可继承并实现 asStructLike() 和 getInputFile() 以支持自定义行类型
- BaseDeleteLoader — 可重写 canCache() 和 getOrLoad() 方法以添加缓存支持（如 Caffeine 缓存）
- RegistryBasedFileWriterFactory<T, S> — 引擎可继承并绑定自己的行类型和 Schema 类型
- FormatModelRegistry — 可注册新的文件格式模型以支持更多存储格式

7.3 版本演进说明

iceberg-data 模块正在经历从 BaseFileWriterFactory 到 RegistryBasedFileWriterFactory 的架构演进。以下类已标记为 @Deprecated，将在 1.12.0 版本中移除：

废弃类	替代方案	废弃版本
BaseFileWriterFactory	RegistryBasedFileWriterFactory	1.11.0
GenericAppenderFactory	GenericFileWriterFactory	1.11.0

第8章 设计亮点与总结

8.1 设计亮点

- 引擎无关的通用抽象：整个模块围绕 Record 通用类型设计，既可以独立使用也可以被 Spark/Flink 等引擎继承扩展，实现了良好的关注点分离。
- 完整的 V2 删除语义支持：DeleteFilter + BaseDeleteLoader 构成了完整的行级删除处理框架，统一支持 Equality Deletes、Position Deletes 和 Deletion Vectors 三种删除机制。
- 格式注册表的插件化设计：通过 FormatModelRegistry + GenericFormatModels 的注册表模式，将文件格式的读写实现与业务逻辑完全解耦，新增格式只需注册不需改代码。
- 渐进式废弃与平滑迁移：新旧两套写入器工厂（BaseFileWriterFactory vs RegistryBasedFileWriterFactory）并存，通过 @Deprecated 注解和文档引导用户迁移，保证了向后兼容性。
- 内置缓存扩展点：BaseDeleteLoader 的 canCache/getOrLoad 保护方法预留了缓存扩展点，上层可以根据实际需要注入 Caffeine 等缓存实现，优化重复删除文件的加载性能。

8.2 设计限制与改进方向

- 单包扁平结构：14 个文件全部位于 org.apache.iceberg.data 同一个包下，随着功能增长可能需要拆分为 reader/writer/delete 等子包。
- 嵌套类型 Equality Delete 支持有限：DeleteFilter.fileProjection() 中有 TODO 注释表明当前不支持嵌套列的 Equality Delete，需要后续完善。
- DV 无缓存支持：Deletion Vector 目前不走缓存路径（BaseDeleteLoader.readDV 直接读取），在多 split 处理同一文件时可能有性能优化空间。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★☆	分层清晰、职责明确，但包结构可以更精细
设计模式运用	★★★★★	Builder、抽象工厂、模板方法、适配器、策略+注册表等模式运用精准
可扩展性	★★★★★	通过抽象类、保护方法和注册表模式预留了丰富的扩展点
API 设计	★★★★★	IcebergGenerics Builder API 简洁直观，符合 Iceberg 整体风格
向后兼容	★★★★★	@Deprecated 标注清晰，新旧 API 并存过渡平滑
文档完整性	★★★★☆	部分类缺少 Javadoc，内部类和包内可见类文档较少

8.4 总结

Apache Iceberg Data 模块是 Iceberg 项目中承上启下的关键数据读写层。它通过精心设计的抽象和模式，在仅 14 个源文件中实现了通用数据读取、三种行级删除处理、多格式文件写入和表迁移等核心能力。模块正在经历从传统工厂模式到注册表驱动模式的架构演进，体现了 Iceberg 社区持续优化和面向未来设计的工程文化。